

Recursion Basics - Coding Assessment

Q1. Write a recursive function to calculate the factorial of a number.

Example: ``factorial(5) => 120``

Q2. Write a recursive function to compute the nth Fibonacci number.

Example: ``fibonacci(6) => 8``

Q3. Given a number ``n``, write a recursive function to return the sum of digits of ``n``.

Example: ``sumDigits(123) => 6``

Q4. Write a recursive function to reverse a string.

Example: ``reverse("hello") => "olleh"``

Q5. Write a recursive function to count how many times a particular character appears in a string.

Example: ``countChar("banana", 'a') => 3``

Q6. Write a recursive function that prints numbers from ``n`` down to ``1``.

Example: ``printReverse(5)`` should print:

```
5
4
3
2
1
```

Q7. Real-world scenario: You are building a file explorer. Each folder can have subfolders. Write a recursive function to print the names of all folders.

Assume a folder structure like:

```
python
folder = {
    "name": "root",
    "subfolders": [
        {"name": "docs", "subfolders": []},
        {"name": "images", "subfolders": [
            {"name": "vacation", "subfolders": []}
        ]}
    ]
}
```

Q8. Real-world scenario: Write a recursive function to calculate the total cost of items in a nested shopping cart.

Assume this structure:

```
python
cart = {
    "price": 10,
    "subitems": [
        {"price": 5, "subitems": []},
        {"price": 2, "subitems": [
            {"price": 1, "subitems": []}
        ]}
    ]
}
```

Q9. Write a recursive function to check if a string is a palindrome.

Example: `isPalindrome("madam") => True`

Q10. Real-world scenario: You are designing a social network tree. Write a recursive function that counts the number of users in the network.

Each user can have friends who also have friends (like a tree):

```
python
user = {
    "name": "Alice",
    "friends": [
        {"name": "Bob", "friends": []},
        {"name": "Charlie", "friends": [
            {"name": "Dave", "friends": []}
        ]}
    ]
}
```

}]]}